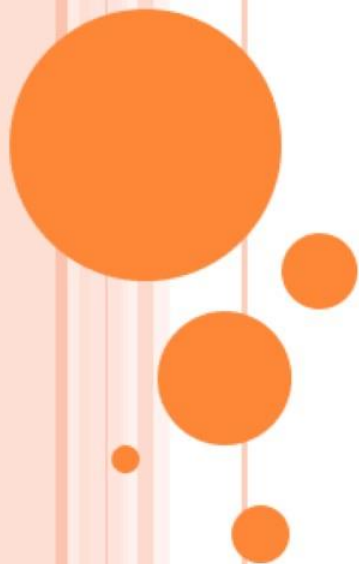


ЭТАПЫ РАБОТЫ КОМПИЛЯТОРА



- Транслятор - это программа, которая переводит исходную программу в эквивалентную ей объектную программу. Если объектный язык представляет собой автокод или некоторый машинный язык, то транслятор называется **компилятором**.
- Автокод очень близок к машинному языку; большинство команд автокода - точное символическое представление команд машины.
- Ассемблер - это программа, которая переводит исходную программу, написанную на автокоде или на языке ассемблера, в объектный (исполняемый) код.
- Компиляторы пишутся как на автокоде, так и на языках высокого уровня. Кроме того, существуют и специальные языки конструирования компиляторов - компиляторы компиляторов.
- **Компилятор компиляторов (КК)** – система, позволяющая генерировать компиляторы; на входе системы - множество грамматик, а на выходе, в идеальном случае, - программа.

Иногда под КК понимают язык программирования, в котором исходная программа - это описание компилятора некоторого языка, а объектная программа - сам компилятор для этого языка.

- **Исходная программа КК** - это просто формализм, служащий для описания компиляторов, содержащий, явно или неявно, описание лексического и синтаксического анализаторов, генератора кодов и других частей создаваемого компилятора.
- **Исходная программа** – текст программы на языке высокого уровня, который должен быть переведен в машинный код.
- **Информационные таблицы** - самостоятельные структуры, заранее заполненные, а также заполняющиеся в ходе лексического анализа и дополняющиеся во время работы.
- **Лексический анализатор** выполняет распознавание лексем языка и замену их соответствующими кодами. Под лексемами понимаются элементарные единицы, входящие в структуру предложения языка, такие как ключевые слова, константы, имена и т.п. Результатом является поток лексем (кодов – ссылок на таблицы), эквивалентный исходному тексту.



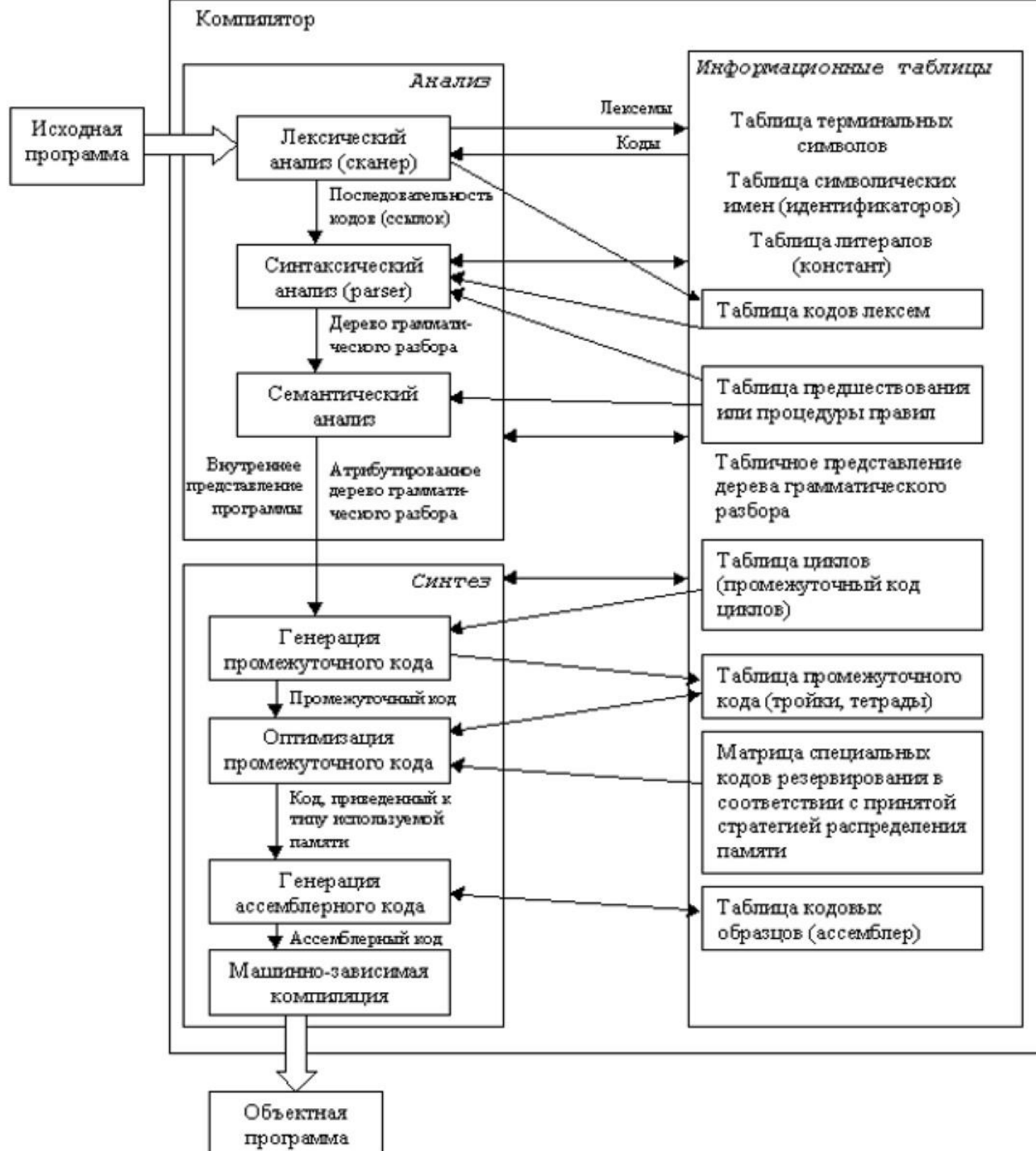
- **Синтаксический анализатор** необходим для того, чтобы выяснить, удовлетворяют ли предложения, из которых состоит исходная программа, правилам грамматики этого языка.
- **Семантический анализ.** На этом этапе осуществляется контроль типа и вида всех идентификаторов и других операндов.
- **Генерация промежуточного кода.** Происходит преобразование исходной программы в промежуточную (например, польскую) форму записи.
- **Оптимизация промежуточного кода** – выделение общих подвыражений и вычисление константных подвыражений.
Фаза оптимизации предназначена для уменьшения избыточности программы по затратам времени и памяти. В зависимости от критериев проектирования транслятора данная фаза обработки программы может исключаться из цикла обработки программы.



- **Распределение памяти.** На этом этапе выделяются конкретные адреса пользователя под переменные, которые генерируются компилятором.
- **Генератор объектного (ассемблерного) кода** – выполняет подстановку кодовых образцов на выходном языке, соответствующих промежуточным кодам программы. Генератору кода могут не требоваться шаблоны, он весь может быть реализован в процедурном виде.
- **Исходная программа** – текст программы на языке высокого уровня (например Паскаль), который должен быть переведен в машинный код.
- **Информационные таблицы** - самостоятельные структуры, заранее заполненные (таблица терминальных символов), а также заполняющиеся в ходе лексического анализа и дополняющиеся во время работы.



- Лексический анализатор выполняет распознавание лексем языка и замену их соответствующими кодами. Под лексемами понимаются элементарные единицы, входящие в структуру предложения языка, такие как ключевые слова, константы, имена и т.п. Правильность задания структуры предложения языка на фазе лексического анализа не выполняется. Результатом является поток лексем (кодов – ссылок на таблицы), эквивалентный исходному тексту.
- **Генератор объектного (ассемблерного) кода** – выполняет подстановку кодовых образцов на выходном языке, соответствующих промежуточным кодам программы.
- **Машинно-зависимая компиляция.** Зависит от того, какие используются регистры. Работа этой процедуры зависит от соглашений, принятых для исполняемой части программы. Например, выделяется базовый регистр для текущей активной записи в стеке.
- На всех этих этапах происходит работа с различного рода таблицами. В частности, для каждого блока (если таковые существуют в языке) идентификаторы, описанные внутри, запоминаются вместе со своими атрибутами.



ГРАММАТИКИ

Грамматика языка программирования является формальным описанием его *синтаксиса* или формы, в которой записаны отдельные предложения программы или вся программа. Грамматика не описывает *семантику* или значения различных предложений. Информация о семантике содержится в программах генерации объектного кода.

В качестве иллюстрации разницы между синтаксисом и семантикой рассмотрим два предложения:

$$I:=J+K \text{ и } I:=X+Y,$$

где X и Y являются действительными переменными,
а I, J, K — целыми переменными.

Эти два предложения имеют одинаковый синтаксис, однако их семантика совершенно различна.



ЛЕКСИЧЕСКИЙ АНАЛИЗ

Лексема (лексическая единица языка) – это структурная единица языка, которая состоит из элементарных символов языка и не содержит в своем составе других структурных единиц языка (идентификаторы, константы, ключевые слова языка, знаки операций и т.п.).

Лексический анализатор (сканер) – это часть компилятора, которая читает исходную программу и выделяет ее в тексте лексемы входного языка.

На вход лексического анализатора поступает текст исходной программы, а выходная информация передается для дальнейшей обработки компилятором на этапе синтаксического анализа и разбора.



ЛЕКСИЧЕСКИЙ АНАЛИЗ ВАЖЕН ДЛЯ ПРОЦЕССА КОМПИЛЯЦИИ ПО НЕСКОЛЬКИМ ПРИЧИНАМ:

- замена в программе идентификаторов, констант, ограничителей и служебных слов лексемами, делает представление программы более удобным для дальнейшей обработки;
- лексический анализ уменьшает длину программы, устраняя из нее исходное представление несуществующих конструкций;
- при изменении кодировки в исходном представлении программы отражается только на лексическом анализаторе.

Выбор конструкции, который будут выделяться как отдельные лексеммы, зависит от языка и от точки зрения разработчиков компилятора.



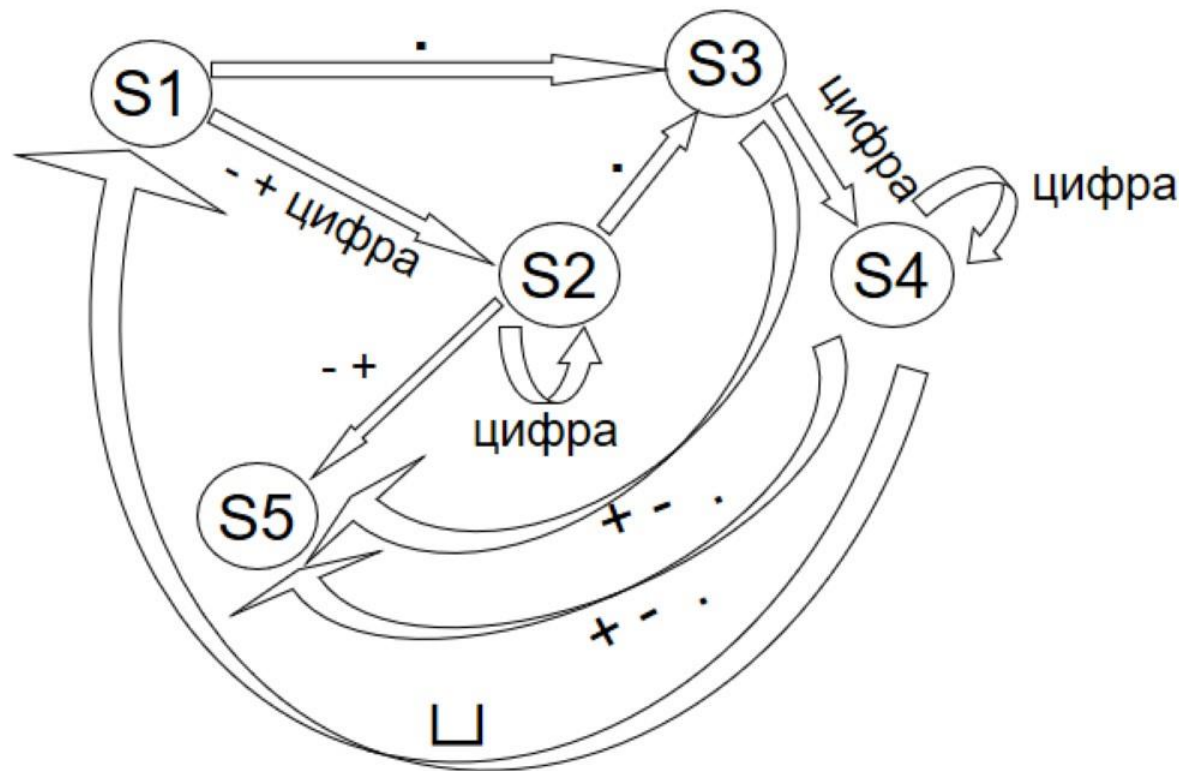
- Лексические анализаторы обычно используются для распознавания элементарных конструкций, синтаксическое описание которых можно выполнить с применением правосторонних грамматик.
- Простота его заключается в том, что лексемы – это единственные нетерминалы, используемые в ходе распознавания, даже в грамматике существуют промежуточные нетерминалы, они исчезают при построении конечного автомата.
- Конечный автомат – устройство для распознавания строк какого-либо языка. Автомат имеет конечное множество состояний, некоторый из которых называют последним.

По мере считывания каждой литеры строки контроль передается от состояния к состоянию в соответствии с заданным множеством переходов.

После считывания последней литеры автомат будет находиться в одном из последних состояний, а строка принадлежит языку, определенного автоматом.



Каждому языку 3 типа соответствует детерминированный конечный автомат, распознающий строки. Написание лексического анализатора частично сводится к построению различных автоматов для распознавания конструкций языка (идентификаторы, числа, ключевые слова).



ПРИ РАБОТЕ СКАНЕРА ИСПОЛЬЗУЮТСЯ ТРИ ТАБЛИЦЫ:

1. Таблица терминальных символов (хранит все ключевые слова и специальные символы, используемые в языке, а также коды, соответствующие каждому символу).
2. Таблица символических имен (заполняется в процессе разбора текста программы и хранит в себе имена идентификаторов).
3. Таблица литералов (хранит в себе литералы: численные и строковые значения, с указанием типов данных и относительных адресов).



Сканер в процессе анализа текста программы выделяет один из элементов текста и сравнивает с каждым терминальным символом.

- Если такой символ найден, то в выходной код передаются код таблицы и спецификатор (номер строки в таблице).
- В случае если этот элемент не является терминальным символом проверяется, является ли он идентификатором (первый символ обычно буква, остальные могут быть либо буквой, либо цифрой), если такой определен, то данный элемент заносится в таблицу символических имен, а к выходному коду добавляется пара численных значений: номер таблицы и спецификатор найденного элемента.
- В случае если такой элемент в таблице уже имеется, то в выходной код заносится номер таблицы и его спецификатор. В таблицу литералов заносят численные, строковые и иные определенные языком значения. При этом распознается тип значения и тут же заполняется относительная таблица адресов.



АЛГОРИТМ РАБОТЫ ЛЕКСИЧЕСКОГО АНАЛИЗАТОРА

- Просматривается входной поток символов программы на исходном языке до обнаружения очередного символа, ограничивающего лексему;
- для выбранной части входного потока выполняется функция распознавания лексемы;
- при успешном распознавании информация о выделенной лексеме заносится в таблицу лексем, и алгоритм возвращается к первому этапу;
- при неуспешном распознавании выдается сообщение об ошибке, а дальнейшие действия зависят от реализации сканера - либо его выполнение прекращается, либо делается попытка распознать следующую лексему (идет возврат к первому этапу алгоритма).
- Работа программы-сканера продолжается до тех пор, пока не будут просмотрены все символы программы на исходном языке из входного потока.



Результатом работы сканера является последовательность кодов лексем.

Каждый код лексемы обычно представляется кодом таблицы и спецификатором. Это позволяет избежать дополнительного поиска по таблицам на следующих этапах трансляции. Например в результате обработки сканером следующего предложения языка Паскаль

FOR I:=1 TO 100 DO Y:=X1

будет получена строка:

<1,06><2,1><1,14><3,1><1,07><3,2><1,08><2,2><1,14><2,3>,

где в угловых скобках пара чисел задает код таблицы и спецификатор.

Таблица выходных символов:

№п.п.	1	2	3	4	5	6	7	8	9	10
Таблица	1	2	1	3	1	3	1	2	1	2
Строка	6	1	14	1	7	2	8	2	14	3



СИНТАКСИЧЕСКИЙ АНАЛИЗ

Синтаксический анализатор – это часть компилятора, которая отвечает за выявление и проверку синтаксических конструкций входного языка. В задачу синтаксического анализа входит:

- поиск и выделение синтаксических конструкций в тексте исходной программы;
- установка типа и проверка правильности каждой синтаксической конструкции;
- представление синтаксических конструкций в виде, удобном для дальнейшей генерации текста результирующей программы.



- Во время синтаксического анализа предложения исходной программы распознаются как языковые конструкции, описываемые исходной грамматикой.
- Процесс заключается в построении дерева грамматического разбора для транслируемых предложений.
- При выполнении синтаксического анализа осуществляется подтверждение того, что входная цепочка символов является программой, а отдельные подцепочки составляют синтаксически правильные программные объекты.
- Вслед за распознаванием отдельных подцепочек осуществляется анализ их семантической корректности на основе накопленной информации, а затем проводится добавление новых объектов в объектную модель программы.



Варианты синтаксических анализаторов

- Имея грамматику входного языка, в первую очередь выполняется ряд формальных преобразований над этой грамматикой.
- После этого происходит проверка, попадает ли полученная грамматика под один из известных классов Кс-языков, для которых существуют линейные распознаватели.
- Если такой класс найден, то можно строить распознаватель.
- Если не найден – то нужно привести грамматику к одному из известных классов.



Методы грамматического разбора разбиваются на два больших класса

нисходящие

и

восходящие

Нисходящие (методы сверху-вниз) начинаются с аксиомы грамматики, с корня дерева и пытаются так его наращивать, чтобы последующие узлы дерева соответствовали синтаксису анализируемого выражения.



+ наиболее удобный

Восходящие (методы снизу-вверх) начинают с элементов предложения (с "листьев") и отыскивают в грамматике какому понятию они соответствуют, т.е. определяют родительскую вершину для этих элементов, и т.д. пока не приходят к корню дерева (аксиоме грамматики).



+ простота

+ высокая скорость выполнения

- отсутствие диагностики



ВОСХОДЯЩИЙ МЕТОД (МЕТОД ПРЕДШЕСТВОВАНИЯ)

- Основан на анализе пар последовательно расположенных операторов исходной программы и решений вопроса о том, который из них должен выполняться первым.
- Анализируемое предложение рассматривается **слева направо** до тех пор, пока не будет найдено подвыражение, операторы которого имеют более высокий уровень операторного предшествования, чем соседние операторы, это подвыражение распознается в терминах правил перехода, используемой грамматики. Процесс продолжается до тех пор, пока не будет достигнут корень дерева, что и будет означать конец грамматического разбора.
- Первым шагом при разработке процессора грамматического разбора, основанного на методе предшествования должно быть установление отношений предшествования между операторами грамматики.



ВОСХОДЯЩИЙ МЕТОД

	Begin	Read	Id	(	)
Begin		<.	<.		
Read				=	
Id					.>
(			< .		=
)					



ВОСХОДЯЩИЙ МЕТОД

- Необходимо чтобы отношения предшествования были заданы однозначно.
- Если для пар отношение предшествования не существует это означает что они не могут находиться рядом ни в каком грамматически правильном предложении.
- Знак = означает что обе лексемы имеют одинаковый уровень предшествования и должна рассматриваться грамматическим процессором в качестве одной конструкции языка.



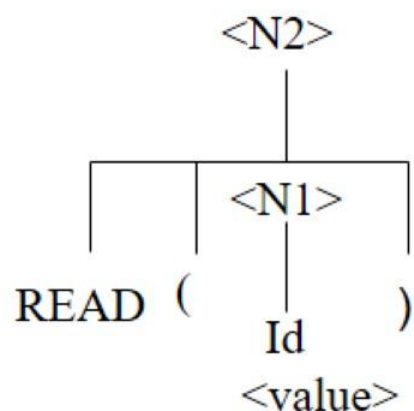
ВОСХОДЯЩИЙ МЕТОД

- Процесс сканирования слева направо продолжается на каждом шаге грамматического разбора до тех пор, пока не определится очередной фрагмент предложения для грамматического распознавания.
- Как только подобный фрагмент выделен, он интерпретируется как некоторый очередной терминальный символ в соответствии с каким-либо правилом грамматики.
- Этот процесс продолжается до тех пор, пока предложение не будет распознано целиком, следует обратить внимание на то, что каждый фрагмент дерева грамматического разбора строится, начиная с конечных узлов вверх в сторону корня дерева.
- Вся информация о грамматике и синтаксических правилах языка содержится в матрице операторного предшествования.



Пример построения дерева восходящим методом

Дерево грамматического разбора для предложения



BEGIN READ (VALUE);

$\langle \cdot \quad \cdot \quad \cdot \quad \cdot \rangle$
 $\quad \quad =$

BEGIN READ ($\langle N1 \rangle$)

$\langle \cdot \quad = \quad = \quad \cdot \rangle$

BEGIN $\langle N2 \rangle$;

$\langle N1 \rangle$

|
Id
{value}



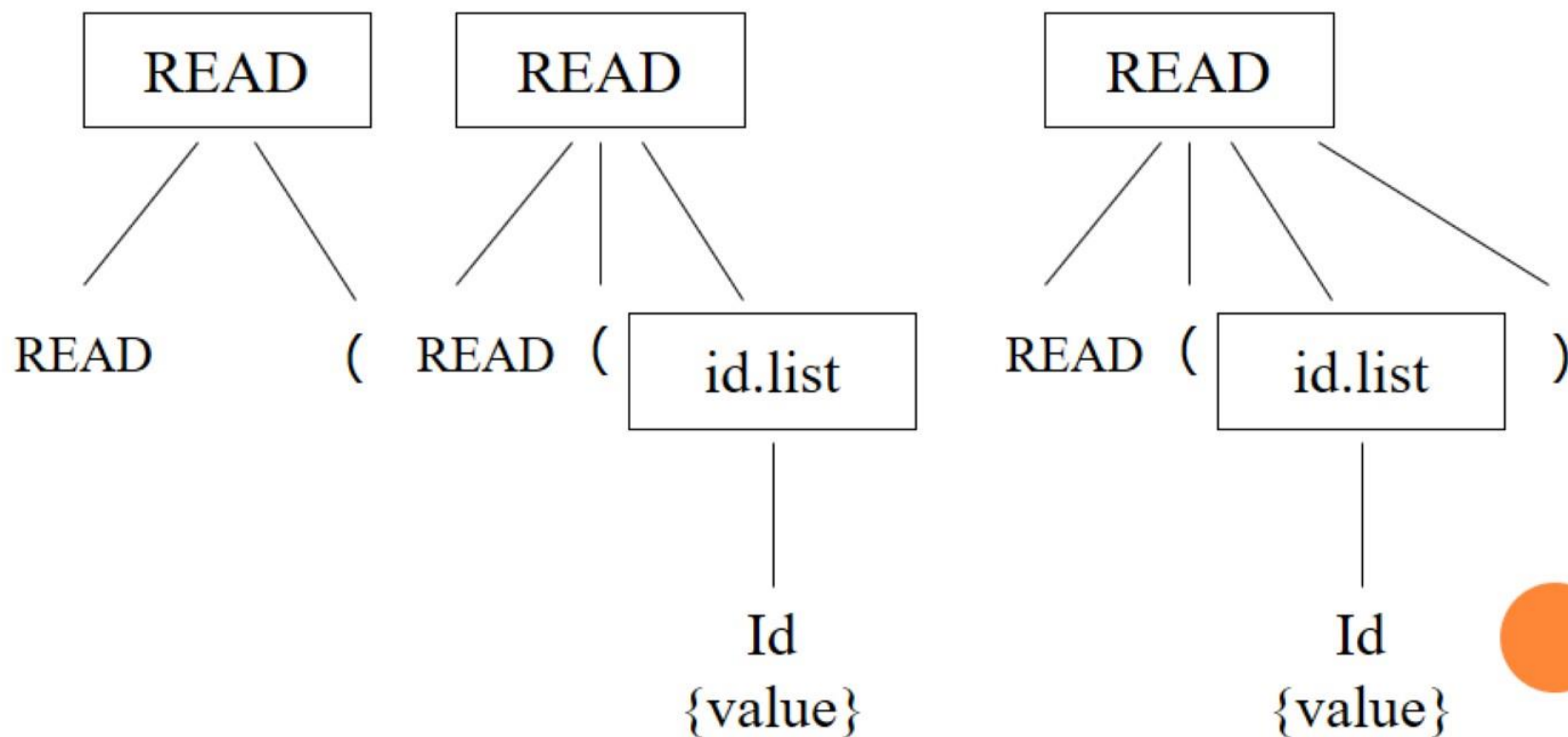
Нисходящий метод (метод рекурсивного спуска)

- Для каждого нетерминала грамматики создается своя процедура, носящая его имя.
- Если такую подцепочку считать не удастся, то процедура завершает свою работу вызовом процедуры обработки ошибки (цепочка не принадлежит языку, и останавливает разбор).
- Если подцепочку удалось найти, то работа процедуры считается нормально завершенной и осуществляется возврат в точку вызова.
- Тело каждой такой процедуры пишется непосредственно по правилам вывода соответствующего нетерминала: для правой части каждого правила осуществляется поиск подцепочки, выводимой из этой правой части.
- Терминалы распознаются самой процедурой, а нетерминалы соответствуют вызовам процедур, носящих их имена.



ПРИМЕР ПОСТРОЕНИЯ ДЕРЕВА НИСХОДЯЩИМ МЕТОДОМ

$\langle \text{read} \rangle ::= \text{READ} (\langle \text{id.list} \rangle)$



НИСХОДЯЩИЙ МЕТОД

- Нисходящий разбор неприменим для грамматик, содержащих левые рекурсии, т.к. один рекурсивный вызов приведет к следующему рекурсивному вызову и т.д., в результате чего образуется бесконечная цепочка рекурсивных вызовов. Для того, чтобы исключить левую рекурсию, в определении идентификатора

$$\langle \text{id.list} \rangle ::= \text{id} \mid \langle \text{id.list} \rangle, \text{id}$$

надо преобразовать в

$$\langle \text{id.list} \rangle ::= \text{id} \{, \text{id} \}$$


Нисходящий метод

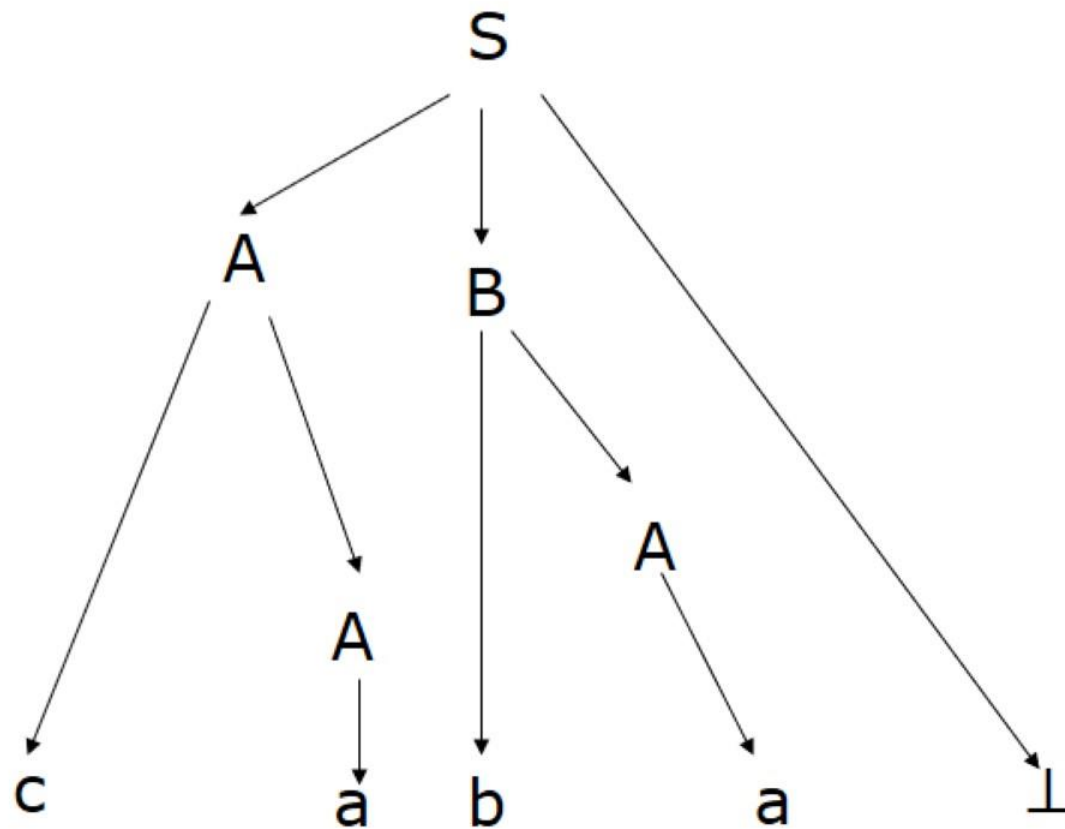
- Обработка нетерминалов может рассматриваться как рекурсивный вызов общих и специальных процедур. Необходимый функциональный минимализм для построения компилятора нисходящего разбора по синтаксическому дереву, состоит из следующих операций:
- 

Нисходящий метод

- Проверка ожидания очередного символа
- Обращение к программе определения очередного терминала
- Успешный или неуспешный выход из анализа нетерминала
- Переход на альтернативную ветвь дерева при неуспешном сравнении
- Переход к стандартной процедуре нейтрализации ошибок в случае неуспеха и отсутствии альтернативных ветвей дерева.



Пример дерева грамматического разбора



$S \rightarrow AB \perp \rightarrow cAB \perp \rightarrow cabA \perp \rightarrow caba \perp$



ВОСХОДЯЩИЙ И НИСХОДЯЩИЙ АЛГОРИТМЫ МОГУТ БЫТЬ ОБОБЩЕНЫ ДЛЯ ДОСТИЖЕНИЯ СЛЕДУЮЩИХ ЦЕЛЕЙ:

- упрощение программы и повышение скорости анализа;
- возможность использования однотипных представлений для спецификации любых задач и программ;
- легкость взаимных преобразований внешней и внутренней формы представления программ и спецификаций;
- выбор эффективной системы кодирования для сокращения запрета на информацию преобразования.



4. ФОРМИРОВАНИЕ ПРОМЕЖУТОЧНОГО КОДА

Возможны различные формы внутреннего представления синтаксических конструкций исходной программы в компиляторе.

Дерево грамматического разбора оказывается неудобным в работе при работе при генерации и оптимизации объектного кода.

Поэтому перед оптимизацией и непосредственно генерацией объектного кода внутреннее представление программы преобразуется в одну из соответствующих форм записи.

Примерами таких форм записи являются:

- обратная польская запись операций;
- тетрады операций;
- триады операций;
- собственно команды ассемблера.



- **Обратная польская запись** - это постфиксная запись операций. Преимуществом ее является то, что все операции записываются непосредственно в порядке их выполнения. Она чрезвычайно эффективна в тех случаях, когда для вычислений используется стек.

- **Тетрады** представляют собой запись операций в форме из четырех составляющих:

$\langle \text{операция} \rangle (\langle \text{операнд1} \rangle, \langle \text{операнд2} \rangle, \langle \text{результат} \rangle)$.

Тетрады используются редко, так как требуют больше памяти для своего представления, чем триады, не отражают взаимосвязи операций и плохо отображаются в команды ассемблера и машинные коды, так как в наборах команд большинства современных машин не встречаются операции с тремя операндами.

- **Триады** представляют собой запись операций в форме из трех составляющих:

$\langle \text{операция} \rangle (\langle \text{операнд1} \rangle, \langle \text{операнд2} \rangle)$,

при этом один или оба операнда могут быть ссылками на другую триаду в том случае, если в качестве операнда данной триады выступает результат выполнения другой триады. Поэтому триады при записи последовательно номеруют для удобства указания ссылок одних триад на другие.

КОМПИЛЯТОРЫ ФОРТРАН

- Фортран (Fortran от FORmula TRANslator - переводчик формул) - первый реализованный язык программирования высокого уровня для машин, построенных по классической схеме фон Неймана. Создан в период с 1954 по 1957 год группой программистов под руководством Джона Бэкуса (John Backus) в корпорации IBM.
- Фортран широко используется в первую очередь для научных и инженерных вычислений. Одно из преимуществ современного Фортрана — большое количество написанных на нём программ и библиотек подпрограмм.
- До 1997 основным производителем компиляторов Fortran для IBM PC совместимых компьютеров была корпорация Microsoft. Впоследствии она отказалась от их разработки в связи с низкой прибыльностью. На данный момент компиляторы поставляет фирма **DEC**, вошедшая в 1998 г. в состав Compaq и вместе с последней в 2002 г. слившаяся с HP.

- Компания DEC поставляет компилятор, интегрированный в среду разработки Digital Visual Fortran, основанную на Microsoft Visual Studio. Наиболее известными продуктами этой линейки являются **FPS 4.0** (Microsoft Fortran Power Station), **DVF 5.0 и 6.0**. Каждый компилятор может поддерживать несколько стандартов Фортрана. Слияния компаний явились причиной того, что последующие продукты появлялась на рынке под торговыми марками Compaq и HP. В настоящее время HP продаёт среду разработки версии 6.6 для Intel/win32. Поддержка Fortran реализована также для всех высокопроизводительных платформ HP.
- Другим крупным поставщиком систем разработки на Fortran является фирма Lahey, предлагающая интегрированные решения для Windows и Linux.
- Долгое время лучшим компилятором Fortran считался компилятор фирмы Watcom, который был выделен в отдельный проект Open Watcom развивающий компилятор на открытой основе.



- Известен и развивается также компилятор фирмы Intel — Intel Fortran Compiler, который позволяет оптимизировать код под платформу Intel ia32 и ia64.
- Среди бесплатных компиляторов Fortran следует выделить компилятор от Sun Microsystems, входящий в состав Sun Studio, который генерирует эффективный код под SPARC, x86 и x86_64 и доступен как для ОС Solaris, так и GNU/Linux.
- Фонд свободного программного обеспечения GNU выпускает открытый компилятор Fortran-77 g77, доступный практически для любой платформы и полностью совместимый с GCC, но не поддерживающий всех языковых конструкций современных стандартов Fortran. Также существует проект g95 по созданию на основе GCC компилятора Fortran 95.
- В России, к сожалению, распространены лишь два компилятора: **Microsoft Fortran Power Station v. 4.0** (Фортран 90) и **Compaq Visual Fortran** или **Digital Visual Fortran** (Фортран 95), работающие в среде Microsoft Developer Studio и обеспечивающие поддержку смешанного программирования на Visual C++ и на Visual Basic.

СПАСИБО ЗА ВНИМАНИЕ!

